



Intrusion Detection System

Saif Indrawes 0239332

Hamzeh Samour 2221066

Zaha AlRashid 0216470

Mohammad Ismail 0231134

Dr. Malik Alessa

Introduction:

An Intrusion Detection System (IDS) is a security mechanism designed to monitor network traffic and identify malicious or suspicious behavior in real time. Unlike traditional firewalls that focus on blocking predefined rules, an IDS provides deeper visibility by analyzing communication patterns and detecting potential intrusions, abuse attempts, or reconnaissance activities such as port scanning and brute-force indicators.

In this project, the IDS is implemented using a hybrid detection approach combining both offline and online verification. The offline detection layer uses a Scalable Bloom Filter to rapidly match IP addresses against a large malicious blocklist stored locally, enabling fast lookup and minimal resource usage. The online layer integrates the AbuseIPDB Threat Intelligence API to validate unknown IPs and extract detailed attack categories and analyst comments when available.

Whenever a suspicious IP is confirmed, the system generates an alert, enriches it with geolocation data (country of origin if retrievable), and stores it in a SQLite database for further inspection. A Flask-based dashboard is also provided to visualize alerts, display scan results, and allow secure start/stop control of the monitoring process without interrupting the system.

The goal of this IDS is educational and defensive — to demonstrate real-time network monitoring, efficient threat lookup, false-alert reduction using a whitelist, and structured logging for security analysis and reporting, making it suitable for IoT and smart-network environments where lightweight and adaptive detection is essential.

System Architecture:

The system follows a modular hybrid IDS architecture consisting of independent yet interconnected components for packet capture, threat intelligence lookup, geolocation enrichment, persistent logging, and dashboard-based control.

1. Traffic Capture Layer

- Network packets are captured live from the *network interface* using **PyShark** LiveCapture, allowing real-time monitoring of all communications passing through the local network.

2. Threat Lookup Engine

- Every extracted source IP (src_ip) is first checked against a whitelist to avoid flagging trusted services such as local gateways and public DNS resolvers.
- If the IP is not whitelisted, it is then matched **offline** using a **Scalable Bloom Filter** loaded from the local blocklist file (malicious.bloom), ensuring very fast lookup with low memory consumption.
- If no offline match is found and the system is in **hybrid mode**, an **online verification** request is sent to the **AbuseIPDB API** to determine if the IP has existing reports and to retrieve attack categories and analyst comments.

3. Threat Enrichment Layer

- If an IP is confirmed as suspicious or malicious, the system attempts to retrieve its country of origin using the **Geocoder** library for additional contextual awareness.

4. Alert & Scan Logging Database

- Confirmed threats generate **alerts** which are stored persistently in a **SQLite database** (ids_alerts.db) inside the alerts table.
- Port scan results (a supporting feature gathered via a full open TCP Scan) are stored in the port_scans table along with service names and port status.

5. Dashboard Control & Visualization Layer

- A **Flask-based dashboard** provides:
 - System status (Running/Stopped/Error)
 - Alert visualization with pagination
 - One-time or periodic port scanning
 - Secure start/stop controls using background threads, so the interface never freezes during monitoring.

Architecture Design Justification:

- **Modularity** ensures each security function operates independently, improving maintainability and clarity.
- **Hybrid mode** balances speed and accuracy by minimizing API calls while verifying only when necessary.
- **Bloom filter lookup** supports IoT environments where lightweight and high-speed decision making is essential.
- **Whitelisting** significantly reduces false alerts on trusted infrastructure IPs.
- **Safe execution** protects the usability of the dashboard, even during continuous network sniffing or repeated scanning tasks.

Experimental Results and Demo Evaluation:

The IDS was tested using live packet capture on the Wi-Fi NIC via PyShark, with IP reputation validated through a whitelist, offline Bloom Filter, and online AbuseIPDB API (**Hybrid Mode**).

Key Results:

- Suspicious IPs were successfully detected and logged as alerts in **SQLite (ids_alerts.db → alerts table)**.
- Whitelisted IPs produced zero false alerts, confirming effective noise reduction.
- Offline lookup was extremely fast and lightweight, suitable for IoT-scale monitoring.
- Online API verification returned accurate attack categories and analyst comments when reports existed.

Demo Outcome:

The Flask dashboard remained responsive without freezing during continuous monitoring, and alerts and optional port-scan results were stored and visualized correctly for academic analysis and reporting.

Conclusion: The system achieved its defensive goals by proving a functional hybrid, real-time, reputation-based intrusion detection pipeline with structured logging and safe dashboard control.

Strengths & Limitations:

Strengths:

- **Real-Time Monitoring** using **PyShark**
- **Fast Offline Reputation Lookup** using **Scalable Bloom Filter**
- **False Alert Reduction** through **IP Whitelisting**
- **Modular Design**, easy to maintain and test
- **Structured Logging** in lightweight **SQLite** with dashboard responsiveness.

Limitations:

- Possible **False Positives** due to Bloom Filter nature (very low probability)
- Detection is primarily **Reputation-Based**, not deep signature or payload analysis
- Limited to scanning **source IP only**, without correlation to full attack context
- Live capture depends on **TShark/Wireshark availability** in the runtime environment

Future Work / Improvements:

- Add **protocol-signature and anomaly analysis**, not reputation only
- Deploy IDS on **edge/IoT devices** for real network defense simulation
- Integrate **ML models** for adaptive threat detection and reduced false positives

Conclusion:

The project successfully implemented a defensive real-time IDS using a hybrid reputation-based detection pipeline on the Wi-Fi NIC. It demonstrated rapid offline threat lookup via Scalable Bloom Filter, online verification using AbuseIPDB API, and false-alert reduction through IP whitelisting. Alerts and port scan results were stored in a lightweight SQLite database and visualized in a responsive Flask dashboard using background threads.

While effective for IoT-scale monitoring, detection remains reputation-focused rather than payload-deep, offering a strong foundation for future expansion into signature, anomaly, and ML-based intrusion analysis.